

Project Scoping Submission - Break The Bot

Team Members -

- Anjali Pai
- Atharv Talnikar
- Nitya Ravi
- Rahul Kulkarni
- Taniksha Datar
- Yashi Chawla

1. Introduction

Large Language Models (LLMs) are increasingly deployed in production systems, where safety failures such as jailbreaks, prompt injections, and policy violations can have serious real-world consequences. Ensuring continuous safety evaluation as models, prompts, and configurations evolve remains a major challenge, as most safety testing is still performed manually or as one-off benchmarks.

This project presents a production-style LLM safety evaluation system that integrates automated safety checks into a modern MLOps workflow. The system continuously evaluates target LLMs against adversarial prompt datasets, scores responses using an LLM-as-a-judge, and computes safety, bias, and refusal-quality metrics such as Attack Success Rate (ASR) and over-refusal.

The evaluation pipeline is orchestrated using Google Cloud Composer and is automatically triggered by configuration changes or scheduled runs, enabling ongoing safety monitoring as models and datasets evolve. All evaluation outputs are stored as versioned artifacts using DVC and Google Cloud Storage to ensure auditability and reproducibility. Safety metrics are exposed through a FastAPI service deployed on Cloud Run and visualized in a Streamlit dashboard, providing near real-time visibility into model safety behavior.

2. Dataset Information

We will use curated datasets of jailbreak and prompt-injection prompts along with custom red-team and attacker-LLM generated examples. Each entry stores the adversarial prompt, target model output (raw or redacted), detector scores, and labels. The dataset supports automated evaluation, safety regression testing, and detector training.

2.1 Data Card:

- **Name:** MLOps-Project-Adversarial-Prompts-v1
- **Sources:**
 1. Curated adversarial prompt datasets (e.g., SALAD)
 2. Configuration-driven prompt sources for continuous evaluation.
 3. Automated attacker LLM generations (templates, paraphrases, mutations).

- **Fields:** `prompt_id`, `prompt_text`, `origin`, `strategy`, `target_model_version`, `target_output_redacted`, `detector_scores`, `human_label`, `run_ts`.
- **Languages:** English.
- **Intended Use:** Continuous LLM safety evaluation, Monitoring safety trends across model versions and configurations and Powering automated evaluation pipelines and safety dashboards.
- **Limitations:** Contains potentially harmful content; redacted versions provided for safe sharing.

2.2 Data Preprocessing:

- **Load Datasets:** Import data from multiple sources such as Salad-Data, AdvBench, and JailbreakBench as specified in configuration.
- **Validation & Standardization:** Ensure consistent field naming and structure across datasets.
- **Data Cleaning:** Remove prompts that are null, empty, or contain malformed entries.
- **Category Normalization:** Map scenario-specific categories (e.g., “Illegal Drugs” → “Illegal Activity”) to a unified taxonomy for consistent downstream evaluation.
- **Feature Engineering:** Add derived columns such as:
`prompt_id`: unique identifier for traceability.
`text_length`: number of words in the prompt.
`size_label`: optional label (short/medium/long) for data slicing.
- **Output Format:** Unified dataframe with columns —
`prompt` | `category` | `prompt_id` | `text_length` | `size_label`.

2.3 Data Rights & Privacy:

- Respect licenses of public datasets; include attribution.
- Production data must be PII-redacted before ingestion.
- Raw harmful outputs stored under access controls; redacted outputs used for dashboards and reports.
- Dataset is internal-only by default; sanitized subsets may be shared externally with disclaimers.

3. Github Repository

Link to repo - <https://github.com/yashichawla/MLOps-Project>

Repository Structure:

```

📁 Repository Structure
MLOps Project/
├── data/      # seed prompts, generated prompts, adversarial examples
├── pipelines/ # evaluation and monitoring pipelines
├── runner/    # evaluator service (FastAPI, executes prompts on target LLMs)
├── judge/     # judge service (FastAPI, scoring unsafe completions)
├── dashboards/ # Grafana dashboards, metrics visualization
├── docs/      # documentation, reports, diagrams
└── tests/     # unit tests, smoke tests, regression suites

```

```
|— .github/
|   |— workflows/ # CI/CD automation (GitHub Actions)
|
|— README.md      # project overview
|— requirements.txt # Python dependencies
|— .gitignore      # ignored files (logs, cache, etc.)
|— docker-compose.yml # container orchestration
```

4. Project Scope

4.1 Problems

- Jailbreak Vulnerabilities: LLMs can be tricked into producing unsafe or harmful outputs via jailbreaks and prompt injections.
- Ad-hoc Safety Testing: Current safety evaluations are often manual, inconsistent, and not embedded in CI/CD pipelines.
- Drift and Regression: Models change over time (new releases, fine-tunes), but teams lack automated monitoring to detect safety regressions.
- Lack of Auditability: Safety evaluations are rarely documented in a repeatable, governance-friendly way.

4.2 Current Solutions

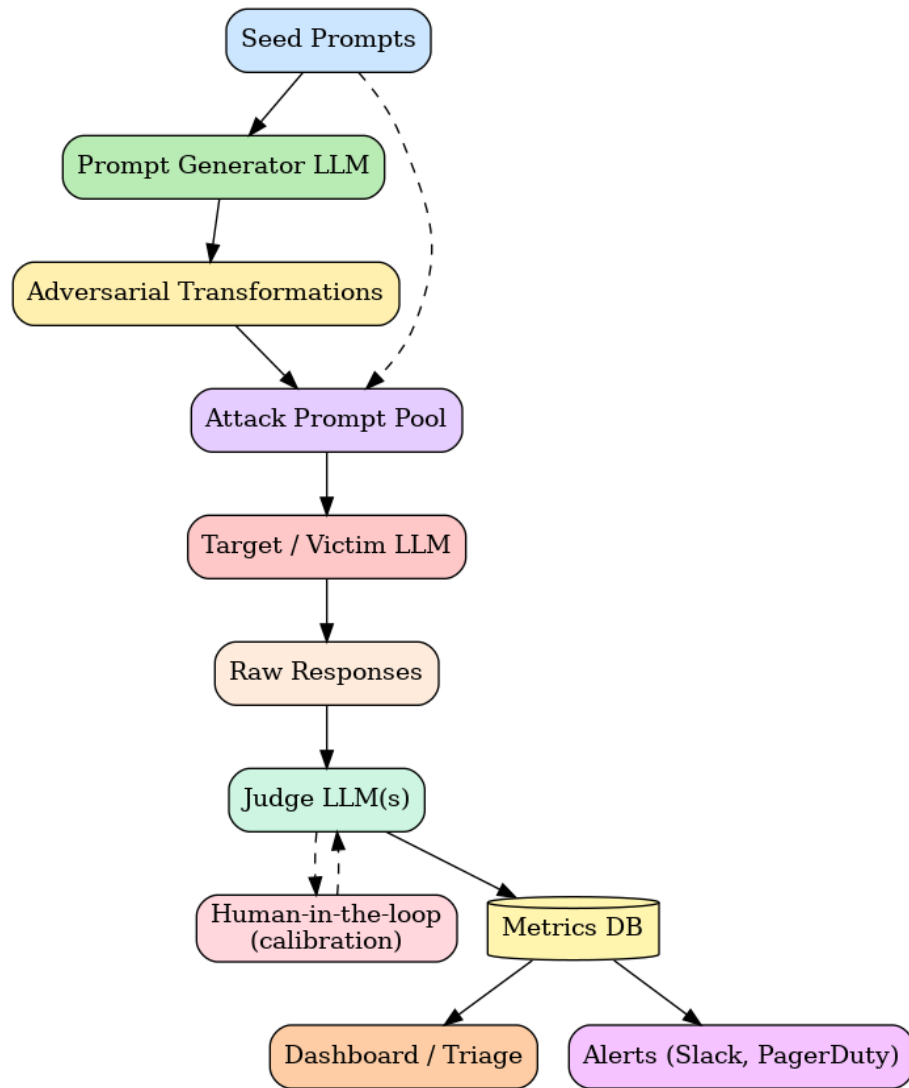
- Manual Red-Teaming: Human testers attempt jailbreaks before deployment, but this is time-consuming, inconsistent, and non-scalable.
- Benchmark Datasets: Provide standardized attack prompts, but results are often static and not tracked over time.
- Closed-Source Safety Evaluations: Proprietary models (e.g., OpenAI's evals) have internal systems, but these are not accessible, auditable, or applicable to open-weight models.

4.3 Proposed Solutions

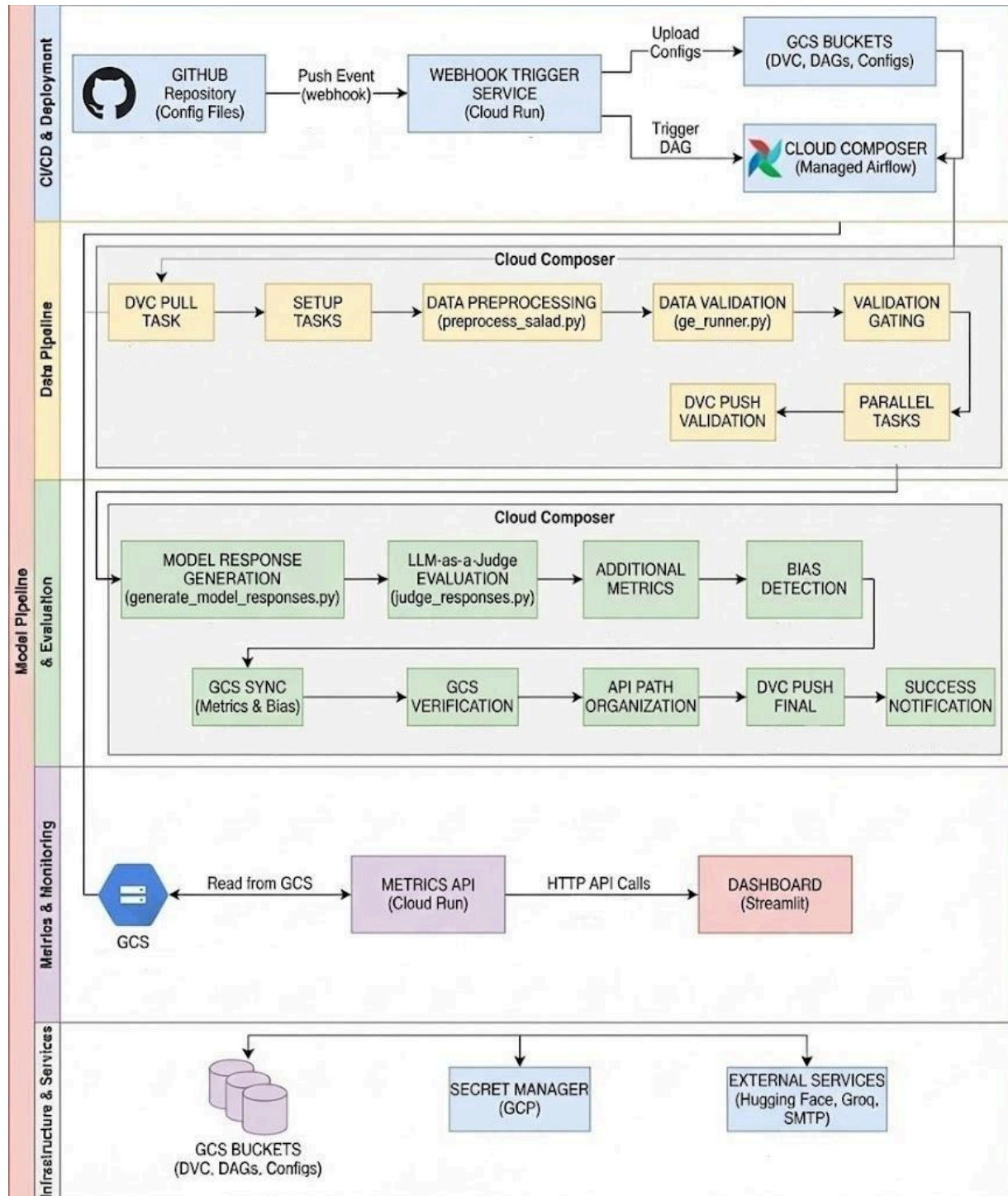
- Automated Safety Pipeline: Build an MLOps system that continuously evaluates open-source LLMs against curated adversarial datasets.
- LLM-as-a-Judge Evaluation: Use judge models to automatically score refusal quality and detect unsafe completions.
- Monitoring & Alerts: Store results, visualize safety metrics (ASR, refusal quality), and trigger alerts when regressions or drift occur.
- Governance Framework: Document safe handling of sensitive data, provide reproducible reports, and ensure transparency in safety evaluations.

5. Current Approach Flowchart and Bottleneck Detection

5.1 Flowchart/ Current Approach



5.2 End-to-End Production-Style Architecture



5.3 Bottlenecks

- Compute & Latency: Running large prompt sets and judge-model scoring can be expensive and slow.
- Dataset Freshness: Static attack sets quickly become outdated; new adversarial prompts must be added.
- Judge Reliability: LLM-as-a-judge may misclassify edge cases or introduce bias into safety scores.
- Drift & Monitoring: Shifts in prompts or model behavior can reduce evaluation accuracy; continuous monitoring is needed.
- Interpretability: High-level metrics (e.g., ASR) need detailed failure examples to explain *why* a model failed.

5.4 Suggested Improvements to Improve Accuracy

- Compute & Latency: Adopt *selective evaluation* or *subset sampling* based on category diversity rather than full-set scoring. Use distributed execution (e.g., Ray or Spark) and cache intermediate model responses to reduce recomputation.
- Dataset Freshness: Implement a *continuous data collection pipeline* that periodically scrapes or generates new adversarial prompts using automatic augmentation or red-teaming frameworks (e.g., [PromptBench](#), [AttackEval](#)).
- Judge Reliability: Use *ensemble judging* — combine outputs from multiple judge models or cross-check with rule-based filters. Include human-in-the-loop verification for ambiguous cases to reduce bias.
- Drift & Monitoring: Integrate *data and model drift detection* modules (e.g., [EvidentlyAI](#), [TFDV](#)) to flag changes in prompt distributions or scoring patterns. Recalibrate the evaluation models periodically.
- Interpretability: Couple quantitative metrics (ASR, harm categories) with *qualitative failure visualization* — such as confusion matrices, example breakdowns, or cluster-based prompt summaries — to better explain failure patterns.

6. Metrics, Objectives, Business Goals

6.1 Metrics

- Attack Success Rate (ASR): % of adversarial prompts that bypass safety.
- Refusal Quality Score: Rated by LLM-as-a-judge for clarity, and completeness of refusals.
- Coverage Metrics: Number and diversity of prompts tested across attack categories.

6.2 Objectives

- Automate red-teaming and safety evaluations in a reproducible pipeline.
- Provide transparent, auditable safety metrics to stakeholders.

6.3 Business Goals

- Reduce the risk of unsafe outputs in deployed applications.
- Improve trustworthiness and compliance of LLM-based systems.

- Enable scalable safety monitoring for future model iterations.

7. Failure Analysis

7.1 Definition of Failures:

- Unsafe Completions: The model outputs harmful or policy-violating responses (e.g., toxic language, PII leakage, successful jailbreaks).
- Weak Refusals: The model refuses to respond but does so ineffectively (e.g., vague refusal, refusal where a safe answer was expected).

7.2 Categorization of Failures:

- PII Leakage: unintended disclosure of personal information.
- Toxic/Harassment: generation of offensive or unsafe content.
- Compliance Refusal Errors: refusing safe queries or misclassifying benign requests.
- Jailbreak Successes: system prompts or safety rules bypassed.
- Hallucinated Refusals: false claim that a safe request is disallowed.

7.3 Analysis Workflow:

- Logging: All failed prompt-response pairs are logged in the metrics database with metadata (model version, timestamp, failure category, judge score).
- Audit Trail: High-severity examples are stored as case studies for compliance reviews and improvement planning.

7.4 Outputs of Failure Analysis:

- Failure Rate by Category: (e.g., 12% jailbreak bypass, 8% weak refusal).
- Trend Analysis: monitoring whether failure rates increase after model updates.
- Recommendations: actionable next steps, such as counter-training, prompt filtering, or refinement of the refusal template.

7.4 Mitigation Strategies:

- PII Leakage: Apply regex-based and embedding-based PII detection before output; reinforce refusal prompts through targeted fine-tuning.
- Toxic/Harassment: Augment training with curated counterexamples and apply toxicity classifiers (e.g., Detoxify) as post-filters.
- Compliance Refusal Errors: Introduce context-aware refusal templates and reinforcement learning from safe completions to improve discernment.
- Jailbreak Successes: Continuously expand red-teaming datasets and employ adversarial training with the latest jailbreak patterns.

- **Hallucinated Refusals:** Refine system prompts to better distinguish between unsafe and safe intents, and integrate confidence-based fallback logic.

8. Deployment Infrastructure

8.1 Objectives

- Enable reproducible, production-style deployment of an automated LLM safety evaluation system.
- Maintain versioned evaluation datasets and metrics artifacts to support auditable, repeatable safety analysis across runs.

8.2 Environments

- **Local Development:**
Local development and testing of pipeline scripts, DAG logic, and API code using Python virtual environments and lightweight local testing.
- **Cloud (GCP):**
Deployed on Google Cloud Platform using Google Cloud Composer for pipeline orchestration, Google Cloud Storage (GCS) for artifact storage, Cloud Build for CI/CD, and Cloud Run for hosting the FastAPI Metrics API.

8.3 Components & Tools

- **Containers:**
Docker images for the Metrics API service, built and deployed using Cloud Build and Cloud Run.
- **Artifact & Data Store:**
Google Cloud Storage (GCS) serves as the centralized storage layer for all evaluation artifacts, including processed datasets, model responses, judge outputs, and computed metrics.
- **Data Versioning:**
DVC with a GCS remote backend to version and track prompt datasets, evaluation outputs, and metrics across pipeline runs, enabling reproducibility and auditability.
- **Pipeline Orchestration:**
Google Cloud Composer (managed Airflow) orchestrates the end-to-end LLM safety evaluation workflow, including preprocessing, evaluation, judging, and metric computation.

- **Observability & Visualization:**

Safety metrics are exposed via a FastAPI service deployed on Cloud Run and visualized through a Streamlit dashboard. Basic service health and request metrics are provided by Cloud Run's built-in monitoring, while pipeline-level observability is handled through Airflow logs and task states.

8.4 Topology

- **Evaluation Pipeline (Batch):**

A Composer-orchestrated pipeline executes prompt preprocessing, target LLM evaluation, LLM-as-a-judge scoring, and metric computation as batch jobs.

- **Metrics Serving Layer (Online):**

A FastAPI Metrics API deployed on Cloud Run reads finalized artifacts from GCS and serves them to downstream consumers, including the Streamlit dashboard.

8.5 CI/CD

- Cloud Build is used to automatically build, test, and deploy the Metrics API container to Cloud Run.
- GitHub webhooks are used to trigger configuration updates and pipeline executions, enabling continuous safety evaluation as models or datasets evolve.

8.6 Secrets & Governance

- **Secrets Management:**

Sensitive credentials (e.g., API keys) are managed using GCP Secret Manager and environment variables, and are never committed to source control.

- **Content Governance:**

Potentially harmful prompt data and model outputs are stored under access controls. Public-facing dashboards and APIs expose only aggregated metrics and sanitized summaries.

8.7 Storage Layout

GCS Buckets / Paths:

- Versioned evaluation artifacts managed by DVC
- Metrics and bias reports consumed by the Metrics API
- Configuration files used to drive evaluation runs

8.8 Performance Notes

- The system is designed for scalable batch evaluation rather than inline inference-time filtering.
- Evaluation throughput and cost are managed by controlling batch sizes, prompt counts, and evaluation frequency through Composer configuration.

9. Monitoring Plan

9.1 Service Observability

- **Pipeline Observability:**
Google Cloud Composer provides task-level visibility, logs, retries, and failure alerts for all pipeline stages.
- **API Observability:**
Cloud Run provides built-in monitoring for request rates, latency, error counts, and service health for the Metrics API.

9.2 Safety & Quality KPIs

- **Attack Success Rate (ASR):**
Measured overall and by category to quantify how often adversarial prompts bypass safety mechanisms.
- **Over-Refusal Metrics:**
Used to measure cases where models unnecessarily refuse safe prompts, balancing safety and usefulness.
- **Coverage Metrics:**
Track the number and distribution of prompts evaluated across categories and prompt lengths.

9.3 Data Quality Monitors

- **Data Validation:**
Great Expectations is used to validate schema consistency, detect missing or malformed prompts, and ensure category balance prior to evaluation.

9.4 Drift & Regression Rules

- **Safety Trend Monitoring:**
Historical evaluation artifacts stored via DVC enable comparison of ASR and over-refusal metrics across runs to identify safety regressions or behavioral drift.
- **Regression Awareness (Advisory):**
Identified regressions are surfaced through dashboards and reports to inform further

investigation, rather than automatically blocking deployments.

9.5 Observability Setup

- **Pipeline Observability:**
Google Cloud Composer (Airflow) provides task-level observability, including execution status, retries, logs, and failure alerts for each stage of the safety evaluation pipeline. This enables visibility into pipeline health and operational reliability.
- **Service Observability:**
The Metrics API is deployed on Cloud Run and **leverages Cloud Run's built-in monitoring** to track request counts, latency, error rates, and service health. Application-level health checks are exposed via a `/health` endpoint to verify service availability and GCS access.
- **Safety Metrics Visualization:**
Safety evaluation results, including Attack Success Rate (ASR), over-refusal, coverage metrics, and bias slices, are visualized through a **Streamlit dashboard** powered by the Metrics API. This provides near real-time visibility into model safety behavior and trends across evaluation runs.
- **Auditability:**
All evaluation outputs are stored as versioned artifacts using DVC and Google Cloud Storage, enabling post-run analysis, historical comparison, and reproducibility.

9.6 Runbooks

- **Documented Analysis Playbooks:**
Markdown-based analysis playbooks describing standard investigation steps for common safety issues such as increases in Attack Success Rate (ASR), elevated over-refusal, or unexpected evaluation failures.
- **Guided Mitigation Workflow:**
Each playbook outlines recommended diagnostic actions (e.g., inspecting affected categories, reviewing prompt samples, comparing historical runs) to support consistent and repeatable analysis rather than automated incident response.
- **Governance & Review Support:**
These playbooks support structured review during model updates or dataset changes, ensuring safety regressions are analyzed in a systematic and auditable manner.

10. Success and Acceptance Criteria

10.1 Success Criteria

- A fully functional pipeline that automatically evaluates LLMs against adversarial prompts.
- Safety metrics (e.g., Attack Success Rate, Refusal Quality) are computed, logged, and visualized.
- The system operates reproducibly, with clear documentation and governance guidelines.

10.2 Acceptance Criteria

- Dashboards clearly display trends in ASR and refusal quality over time.
- CI/CD integration blocks deployment if safety metrics fall below defined thresholds.
- Stakeholders (e.g., instructors, evaluators) can audit results and reproduce evaluations using the provided framework.

11. Timeline Planning



Week 1 – Foundations

- Set up GitHub repository with structure (/data, /pipelines, /runner, /judge, /dashboards).
- Define governance and PII redaction policy.

- Collect & version initial seed prompts.
- Configure DVC for dataset versioning.

Week 2 – Prompt Generator LLM

- Build generator service (FastAPI/Python).
- Generate paraphrased and roleplay-based variants of seed prompts.
- Store outputs in prompt pool with metadata (seed_id, generator model, transformations applied).

Week 3 – Adversarial Transformations & Prompt Pool

- Implement obfuscation, encoding, context injection, roleplay personas.
- Create Attack Prompt Pool (versioned, tagged) in Postgres/DVC.
- Add prompt categories (info disclosure, toxic, compliance bypass, etc.).

Week 4 – Runner / Target LLM Integration

- Build evaluator service: pull prompts → call target LLM → log responses.
- Store raw responses in encrypted .
- Log metadata (model version, parameters) in Metrics DB.

Week 5 – Judge LLM

- Deploy judge service with JSON schema: {violation, category, severity, exploitability, confidence}.
- Run first judge calibration with ~200 labeled samples.
- Log judge scores alongside raw responses.

Week 6 – Human-in-the-loop & Calibration

- Build annotation UI (Streamlit/Label Studio).
- Label 500+ prompt-response pairs for calibration.
- Evaluate judge precision/recall against human gold labels.
- Route low-confidence outputs to human annotators.

Week 7 – Metrics DB & Dashboard Prototype

- **Safety Overview (Streamlit):** A Streamlit dashboard that consumes safety metrics from a FastAPI service to visualize Attack Success Rate (ASR), over-refusal, and category-wise bias trends.
- **Evaluation Summary:** A Streamlit dashboard backed by a FastAPI Metrics API that presents aggregated safety evaluation results across models and runs for easy comparison.

Week 8 – Failure Analysis, Clustering & Regression Tests

- Identify recurring jailbreak strategies.
- Add regression tests to CI.

Week 9 – Scaling, Infra & CI Gates

- Dockerize runner + judge APIs.

- Add monitoring for throughput/latency.

Week 10 – Final Validation & Reporting

- Run hidden test set for unbiased evaluation.
- Validate regression suite & CI/CD gates.
- Produce final dashboards, runbooks, ethics checklist.
- Deliver final presentation & project report.

12. Additional Information

12.1 Ethical and Responsible Use

- Purpose Limitation: The pipeline is designed strictly for *evaluation and safety monitoring*, not for building or disseminating harmful prompts.
- Content Handling: All adversarial data will be stored in secure, access-restricted locations. Publicly shared results will use redacted or summarized forms.
- Transparency: Reports and dashboards will document methodology and evaluation governance, enabling auditability without exposing harmful content.

12.2 Future Extensions

- Multilingual Expansion: Incorporate adversarial prompts in other languages (e.g., Hindi, Spanish, Mandarin) to cover global deployment scenarios.
- Adaptive Attacks: Integrate attacker-LLMs that evolve prompt obfuscations over time, simulating real-world adversaries.
- Cross-Model Benchmarking: Extend pipeline to evaluate both open-source and closed-weight APIs (e.g., GPT-4, Claude) for comparative safety metrics.

12.3 Long-Term Vision

This project aims not just to test models today, but to create a repeatable, auditable framework for continuous LLM safety assurance. Over time, it could evolve into a plug-and-play evaluation service for organizations deploying LLMs in sensitive domains such as finance, healthcare, and education.